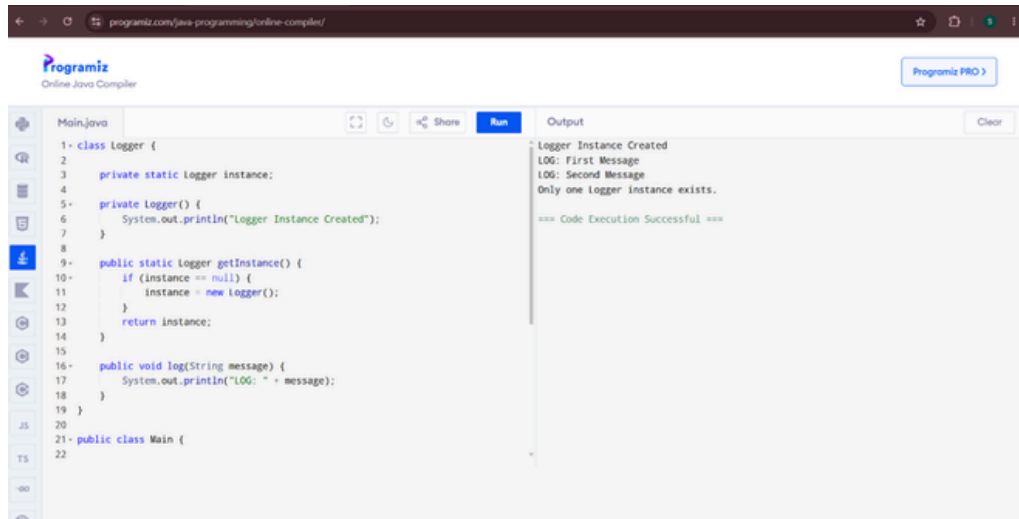


WEEK 1 ASSIGNMENT

DESIGN PATTERNS

1. Singleton Pattern Example (Logger)



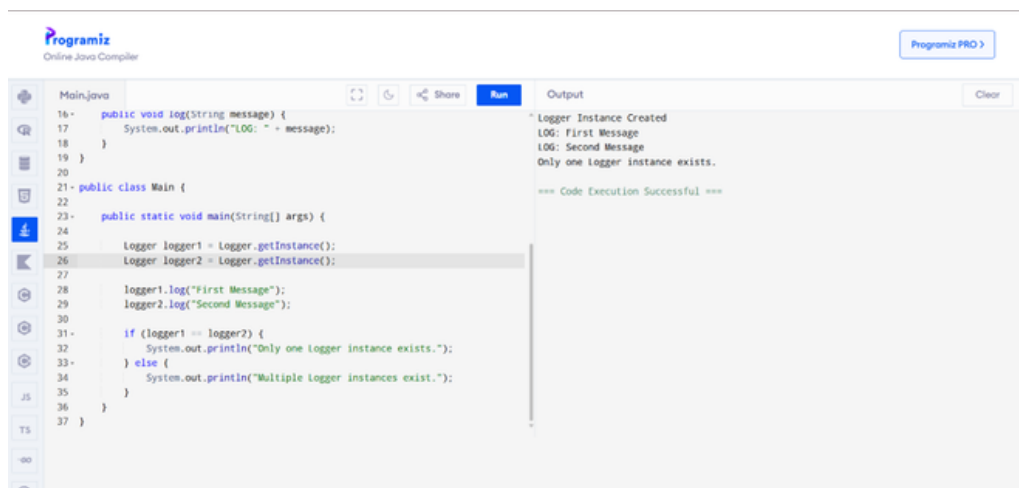
The screenshot shows the Programiz Online Java Compiler interface. The code editor contains the following Java code:

```
1- class Logger {
2
3-     private static Logger instance;
4
5-     private Logger() {
6-         System.out.println("Logger Instance Created");
7-     }
8
9-     public static Logger getInstance() {
10-         if (instance == null) {
11-             instance = new Logger();
12-         }
13-         return instance;
14-     }
15
16-     public void log(String message) {
17-         System.out.println("LOG: " + message);
18-     }
19- }
20
21- public class Main {
22
```

The output window displays the following text:

```
Logger Instance Created
LOG: First Message
LOG: Second Message
Only one Logger instance exists.

=== Code Execution Successful ===
```



The screenshot shows the Programiz Online Java Compiler interface. The code editor contains the following Java code:

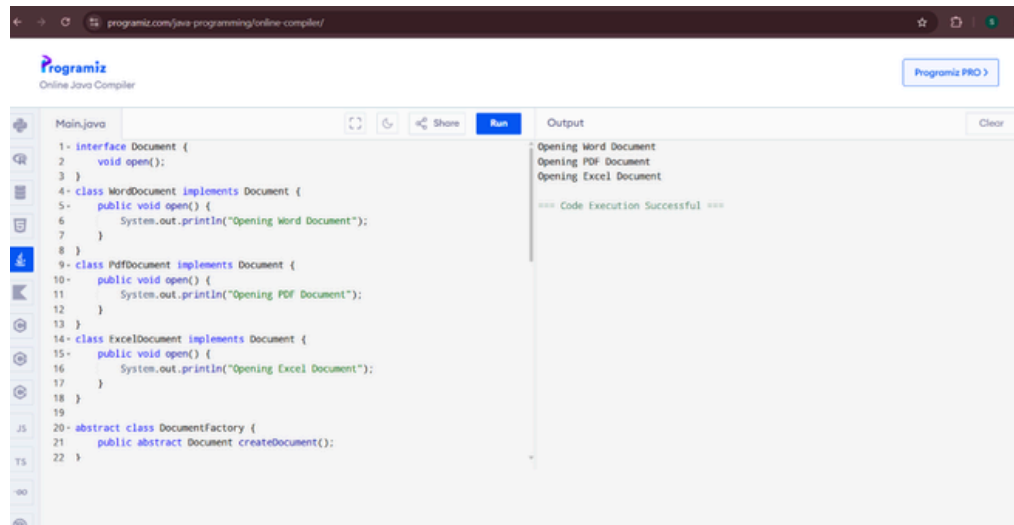
```
16-     public void log(String message) {
17-         System.out.println("LOG: " + message);
18-     }
19- }
20
21- public class Main {
22
23-     public static void main(String[] args) {
24-         Logger logger1 = Logger.getInstance();
25-         Logger logger2 = Logger.getInstance();
26-
27-         logger1.log("First Message");
28-         logger2.log("Second Message");
29-
30-         if (logger1 == logger2) {
31-             System.out.println("Only one Logger instance exists.");
32-         } else {
33-             System.out.println("Multiple Logger instances exist.");
34-         }
35-     }
36- }
37- }
```

The output window displays the following text:

```
Logger Instance Created
LOG: First Message
LOG: Second Message
Only one Logger instance exists.

=== Code Execution Successful ===
```

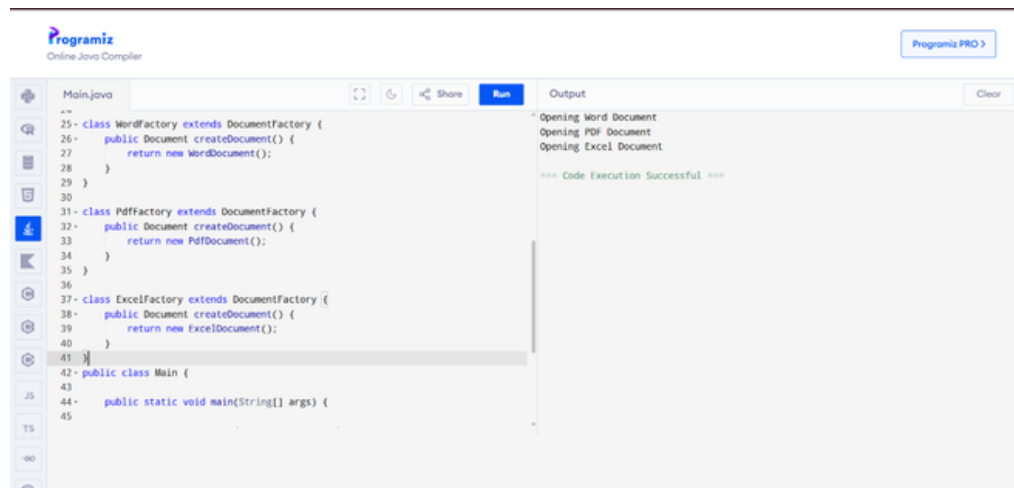
2. FactoryMethodPatternExample.java



```
1- interface Document {
2-     void open();
3- }
4- class WordDocument implements Document {
5-     public void open() {
6-         System.out.println("Opening Word Document");
7-     }
8- }
9- class PdfDocument implements Document {
10-     public void open() {
11-         System.out.println("Opening PDF Document");
12-     }
13- }
14- class ExcelDocument implements Document {
15-     public void open() {
16-         System.out.println("Opening Excel Document");
17-     }
18- }
19-
20- abstract class DocumentFactory {
21-     public abstract Document createDocument();
22- }
```

Output

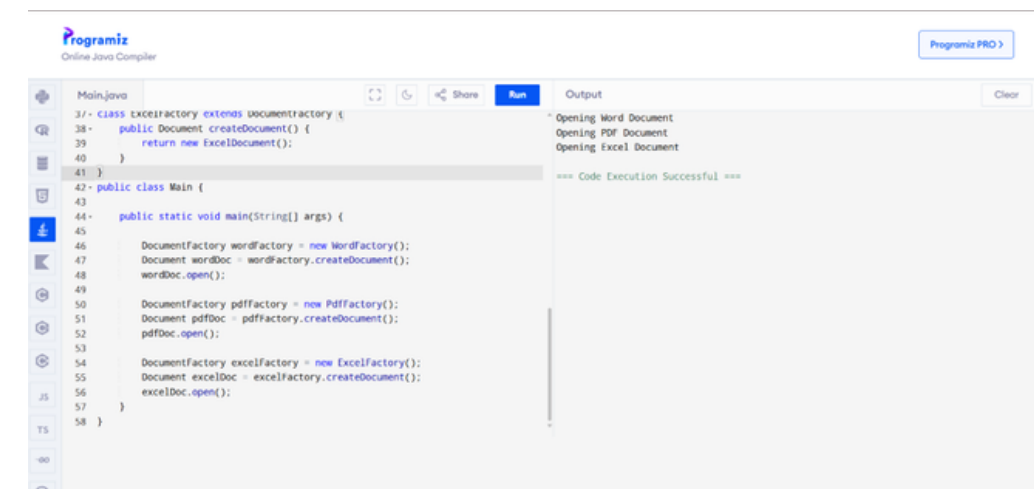
```
Opening Word Document
Opening PDF Document
Opening Excel Document
=== Code Execution Successful ===
```



```
25- class WordFactory extends DocumentFactory {
26-     public Document createDocument() {
27-         return new WordDocument();
28-     }
29- }
30-
31- class PdfFactory extends DocumentFactory {
32-     public Document createDocument() {
33-         return new PdfDocument();
34-     }
35- }
36-
37- class ExcelFactory extends DocumentFactory {
38-     public Document createDocument() {
39-         return new ExcelDocument();
40-     }
41- }
42-
43- public class Main {
44-     public static void main(String[] args) {
45-         // ...
46-     }
47- }
```

Output

```
Opening Word Document
Opening PDF Document
Opening Excel Document
=== Code Execution Successful ===
```

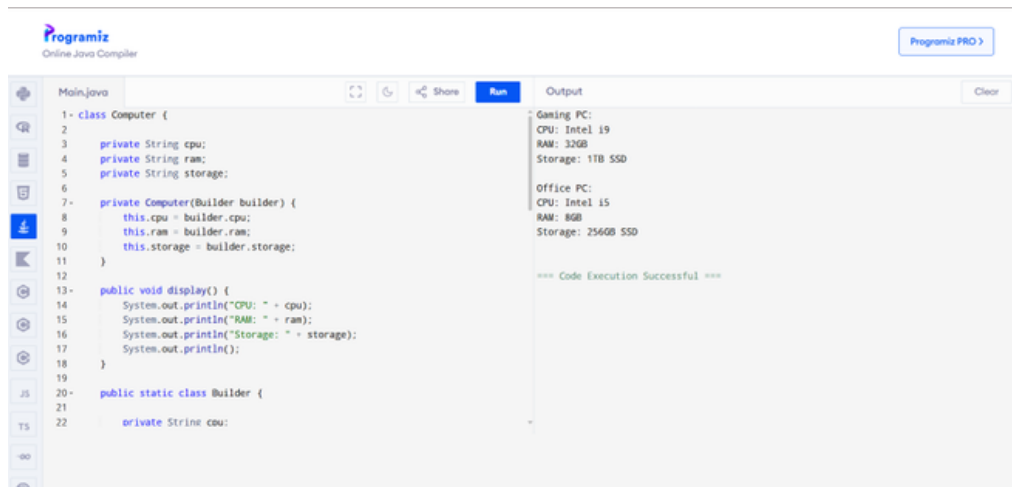


```
37- class ExcelFactory extends DocumentFactory {
38-     public Document createDocument() {
39-         return new ExcelDocument();
40-     }
41- }
42-
43- public class Main {
44-     public static void main(String[] args) {
45-         DocumentFactory wordFactory = new WordFactory();
46-         Document wordDoc = wordFactory.createDocument();
47-         wordDoc.open();
48-
49-         DocumentFactory pdfFactory = new PdfFactory();
50-         Document pdfDoc = pdfFactory.createDocument();
51-         pdfDoc.open();
52-
53-         DocumentFactory excelFactory = new ExcelFactory();
54-         Document excelDoc = excelFactory.createDocument();
55-         excelDoc.open();
56-     }
57- }
```

Output

```
Opening Word Document
Opening PDF Document
Opening Excel Document
=== Code Execution Successful ===
```

3. Builder Pattern



The screenshot shows the Programiz Online Java Compiler interface. The code in Main.java defines a Computer class with private attributes cpu, ram, and storage. It includes a constructor that takes a Builder object and a display() method that prints the attributes. A static Builder class is also defined with a private attribute cpu.

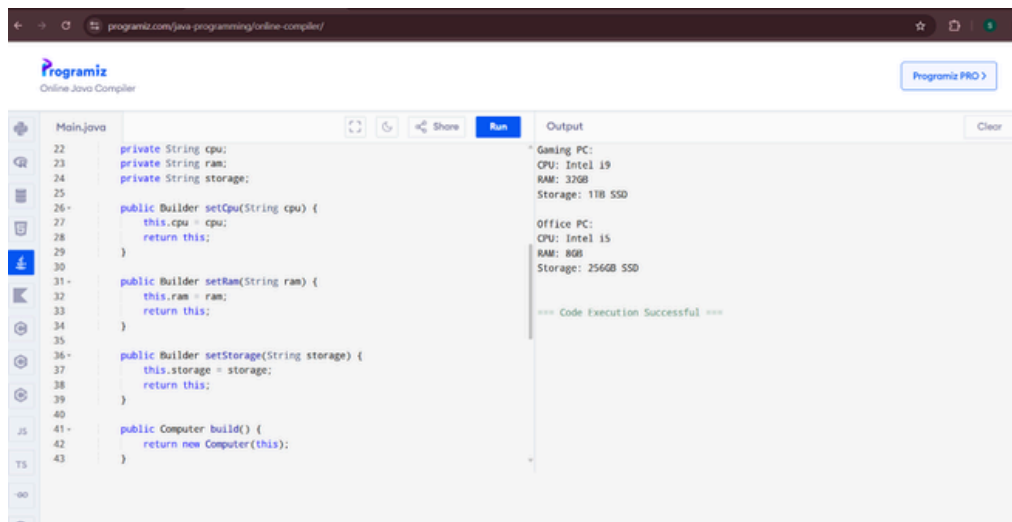
```
1 class Computer {
2
3     private String cpu;
4     private String ram;
5     private String storage;
6
7     private Computer(Builder builder) {
8         this.cpu = builder.cpu;
9         this.ram = builder.ram;
10        this.storage = builder.storage;
11    }
12
13    public void display() {
14        System.out.println("CPU: " + cpu);
15        System.out.println("RAM: " + ram);
16        System.out.println("Storage: " + storage);
17    }
18
19
20    public static class Builder {
21
22        private String cpu;
```

The output window shows the results of running the code, displaying the configuration for a Gaming PC and an Office PC.

```
Gaming PC:
CPU: Intel i9
RAM: 32GB
Storage: 1TB SSD

Office PC:
CPU: Intel i5
RAM: 8GB
Storage: 256GB SSD

=== Code Execution Successful ===
```



The screenshot shows the Programiz Online Java Compiler interface. The code in Main.java defines a Computer class with private attributes cpu, ram, and storage. It includes a constructor that takes a Builder object and a display() method that prints the attributes. A static Builder class is also defined with private attributes cpu, ram, and storage, and setter methods setCpu(), setRam(), and setStorage().

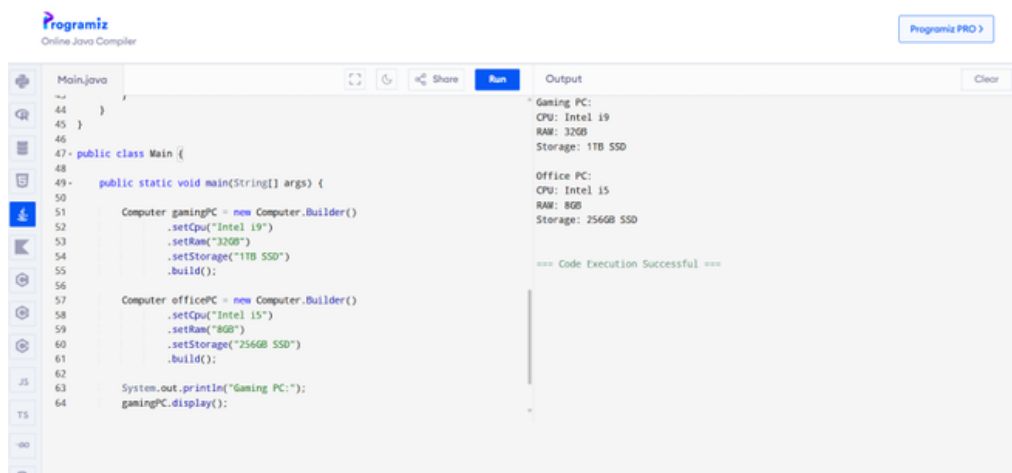
```
22     private String cpu;
23     private String ram;
24     private String storage;
25
26     public Builder setCpu(String cpu) {
27         this.cpu = cpu;
28         return this;
29     }
30
31     public Builder setRam(String ram) {
32         this.ram = ram;
33         return this;
34     }
35
36     public Builder setStorage(String storage) {
37         this.storage = storage;
38         return this;
39     }
40
41     public Computer build() {
42         return new Computer(this);
43     }
```

The output window shows the results of running the code, displaying the configuration for a Gaming PC and an Office PC.

```
Gaming PC:
CPU: Intel i9
RAM: 32GB
Storage: 1TB SSD

Office PC:
CPU: Intel i5
RAM: 8GB
Storage: 256GB SSD

=== Code Execution Successful ===
```



The screenshot shows the Programiz Online Java Compiler interface. The code in Main.java defines a Computer class with private attributes cpu, ram, and storage. It includes a constructor that takes a Builder object and a display() method that prints the attributes. A static Builder class is also defined with private attributes cpu, ram, and storage, and setter methods setCpu(), setRam(), and setStorage(). A Main class is also defined with a main() method that creates and displays two Computer objects.

```
44     }
45 }
46
47 public class Main {
48
49     public static void main(String[] args) {
50
51         Computer gamingPC = new Computer.Builder()
52             .setCpu("Intel i9")
53             .setRam("32GB")
54             .setStorage("1TB SSD")
55             .build();
56
57         Computer officePC = new Computer.Builder()
58             .setCpu("Intel i5")
59             .setRam("8GB")
60             .setStorage("256GB SSD")
61             .build();
62
63         System.out.println("Gaming PC:");
64         gamingPC.display();
```

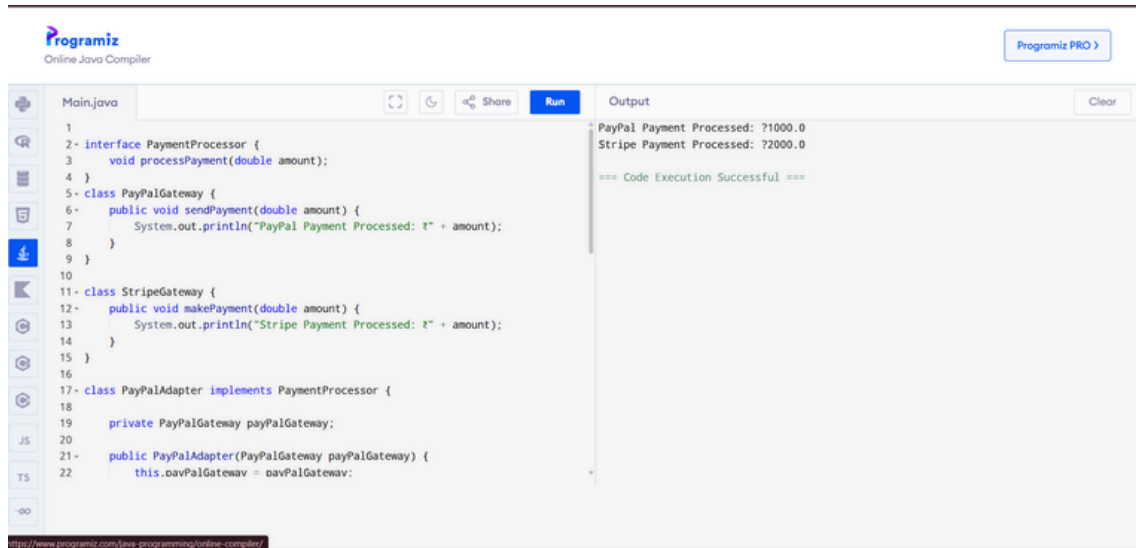
The output window shows the results of running the code, displaying the configuration for a Gaming PC and an Office PC.

```
Gaming PC:
CPU: Intel i9
RAM: 32GB
Storage: 1TB SSD

Office PC:
CPU: Intel i5
RAM: 8GB
Storage: 256GB SSD

=== Code Execution Successful ===
```

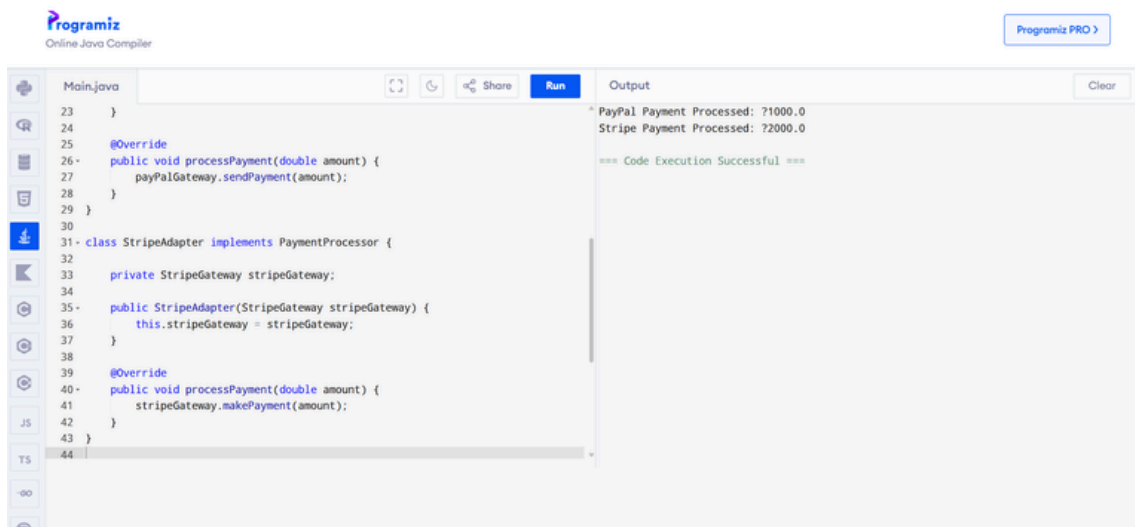
4. Adapter Pattern



```
1 interface PaymentProcessor {
2     void processPayment(double amount);
3 }
4
5 class PayPalGateway {
6     public void sendPayment(double amount) {
7         System.out.println("PayPal Payment Processed: ₹" + amount);
8     }
9 }
10
11 class StripeGateway {
12     public void makePayment(double amount) {
13         System.out.println("Stripe Payment Processed: ₹" + amount);
14     }
15 }
16
17 class PayPalAdapter implements PaymentProcessor {
18     private PayPalGateway paypalGateway;
19
20     public PayPalAdapter(PayPalGateway paypalGateway) {
21         this.paypalGateway = paypalGateway;
22     }
```

Output

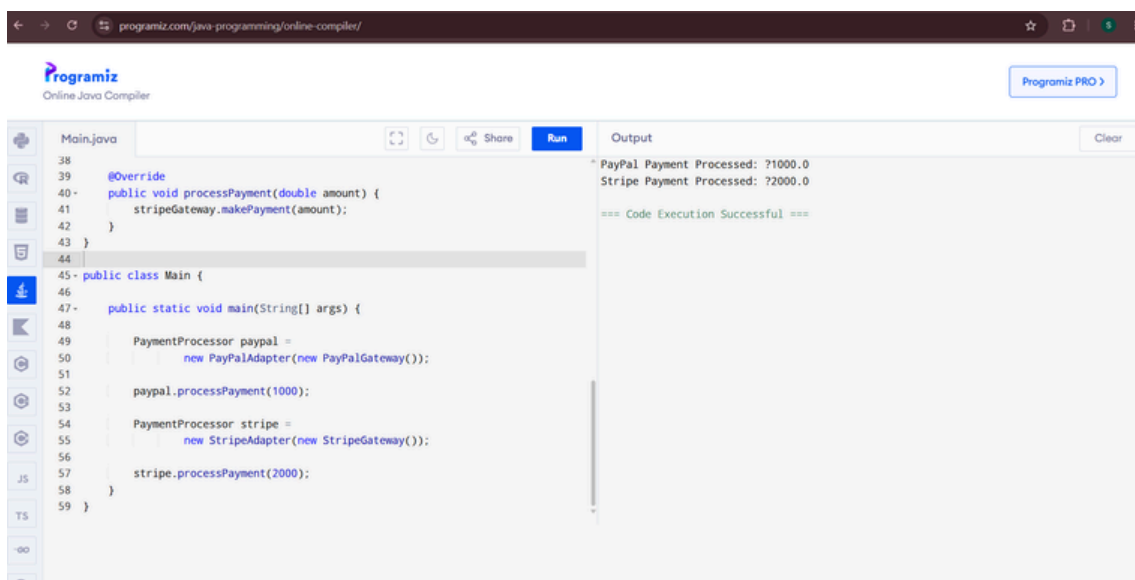
```
PayPal Payment Processed: ₹1000.0
Stripe Payment Processed: ₹2000.0
=== Code Execution Successful ===
```



```
23 }
24
25 @Override
26 public void processPayment(double amount) {
27     paypalGateway.sendPayment(amount);
28 }
29 }
30
31 class StripeAdapter implements PaymentProcessor {
32     private StripeGateway stripeGateway;
33
34     public StripeAdapter(StripeGateway stripeGateway) {
35         this.stripeGateway = stripeGateway;
36     }
37
38 @Override
39 public void processPayment(double amount) {
40     stripeGateway.makePayment(amount);
41 }
42 }
43
44 }
```

Output

```
PayPal Payment Processed: ₹1000.0
Stripe Payment Processed: ₹2000.0
=== Code Execution Successful ===
```



```
38
39 @Override
40 public void processPayment(double amount) {
41     stripeGateway.makePayment(amount);
42 }
43 }
44
45 public class Main {
46     public static void main(String[] args) {
47         PaymentProcessor paypal =
48             new PayPalAdapter(new PayPalGateway());
49         paypal.processPayment(1000);
50
51         PaymentProcessor stripe =
52             new StripeAdapter(new StripeGateway());
53         stripe.processPayment(2000);
54     }
55 }
56
57
58
59 }
```

Output

```
PayPal Payment Processed: ₹1000.0
Stripe Payment Processed: ₹2000.0
=== Code Execution Successful ===
```

5.Decorator Pattern

Programiz
Online Java Compiler

Programiz PRO >

Main.java

1

2- interface Notifier {

3 void send(String message);

4 }

5

6- class EmailNotifier implements Notifier {

7

8 @Override

9- public void send(String message) {

10 System.out.println("Sending Email: " + message);

11 }

12 }

13

14- abstract class NotifierDecorator implements Notifier {

15

16 protected Notifier notifier;

17

18- public NotifierDecorator(Notifier notifier) {

19 this.notifier = notifier;

20 }

21

22 @Override

Output

Clear

Email Notification:
Sending Email: System Update

Email + SMS Notification:
Sending Email: System Update
Sending SMS: System Update

Email + SMS + Slack Notification:
Sending Email: System Update
Sending SMS: System Update
Sending Slack Message: System Update

=== Code Execution Successful ===

Programiz
Online Java Compiler

Programiz PRO >

Main.java

20

21

22 @Override

23- public void send(String message) {

24 notifier.send(message);

25 }

26 }

27

28- class SMSNotifierDecorator extends NotifierDecorator {

29

30 public SMSNotifierDecorator(Notifier notifier) {

31 super(notifier);

32 }

33

34 @Override

35- public void send(String message) {

36 super.send(message);

37 System.out.println("Sending SMS: " + message);

38 }

39 }

40

41- class SlackNotifierDecorator extends NotifierDecorator {

Output

Clear

Email Notification:
Sending Email: System Update

Email + SMS Notification:
Sending Email: System Update
Sending SMS: System Update

Email + SMS + Slack Notification:
Sending Email: System Update
Sending SMS: System Update
Sending Slack Message: System Update

=== Code Execution Successful ===

Programiz
Online Java Compiler

Programiz PRO >

Main.java

41- class SlackNotifierDecorator extends NotifierDecorator {

42

43 public SlackNotifierDecorator(Notifier notifier) {

44 super(notifier);

45 }

46

47 @Override

48- public void send(String message) {

49 super.send(message);

50 System.out.println("Sending Slack Message: " + message);

51 }

52 }

53

54- public class Main {

55

56 public static void main(String[] args) {

57

58 System.out.println("Email Notification:");

59 Notifier emailNotifier = new EmailNotifier();

60 emailNotifier.send("System Update");

61

62 System.out.println("\nEmail + SMS Notification:");

63

Output

Clear

Email Notification:
Sending Email: System Update

Email + SMS Notification:
Sending Email: System Update
Sending SMS: System Update

Email + SMS + Slack Notification:
Sending Email: System Update
Sending SMS: System Update
Sending Slack Message: System Update

=== Code Execution Successful ===

Main.java

52 }

53

54 public class Main {

55

56 public static void main(String[] args) {

57

58 System.out.println("Email Notification:");

59 Notifier emailNotifier = new EmailNotifier();

60 emailNotifier.send("System Update");

61

62 System.out.println("\nEmail + SMS Notification:");

63 Notifier smsNotifier =

64 new SMSNotifierDecorator(new EmailNotifier());

65 smsNotifier.send("System Update");

66

67 System.out.println("\nEmail + SMS + Slack Notification:");

68 Notifier multiNotifier =

69 new SlackNotifierDecorator(

70 new SMSNotifierDecorator(

71 new EmailNotifier());

72

73 multiNotifier.send("System Update");

Run

Share

Output

Clear

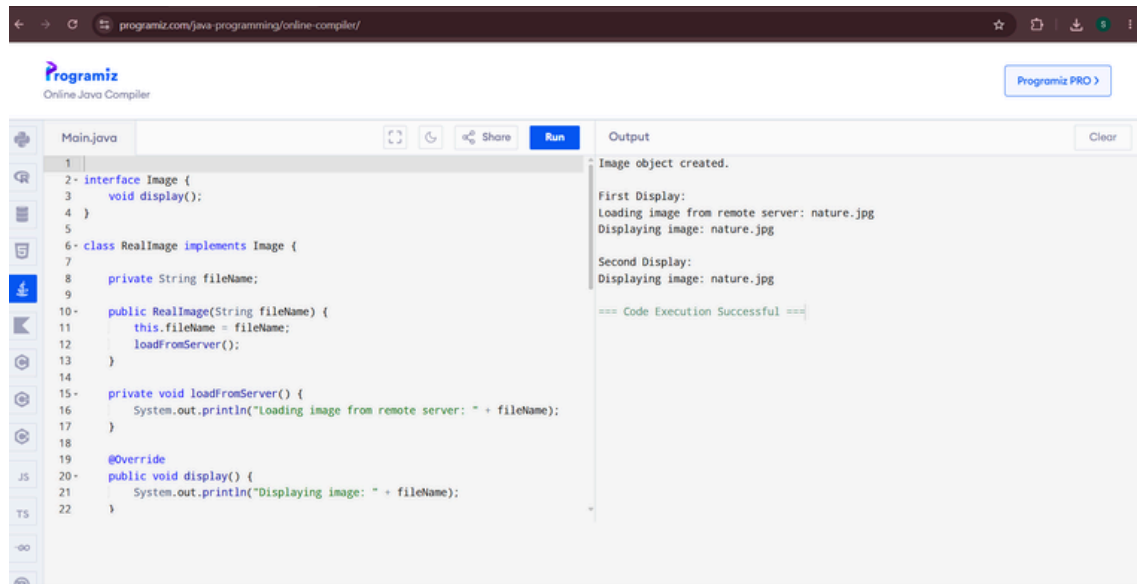
Email Notification:
Sending Email: System Update

Email + SMS Notification:
Sending Email: System Update
Sending SMS: System Update

Email + SMS + Slack Notification:
Sending Email: System Update
Sending SMS: System Update
Sending Slack Message: System Update

*** Code Execution Successful ***

6.Proxy Pattern

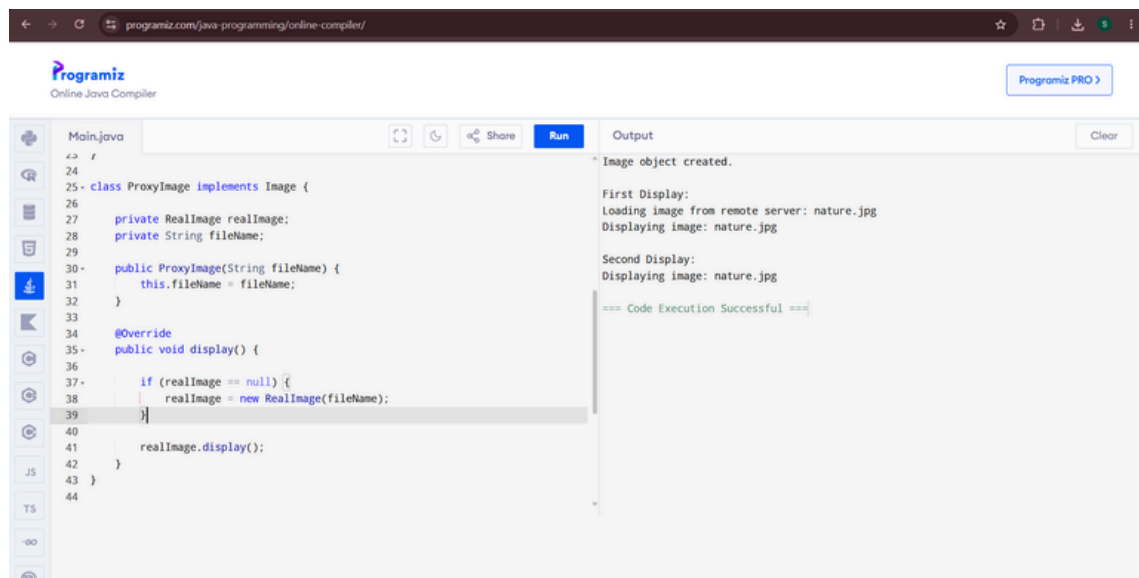


The screenshot shows the Programiz Online Java Compiler interface. The code editor displays the following Java code:

```
1  
2- interface Image {  
3   void display();  
4 }  
5  
6- class RealImage implements Image {  
7  
8   private String fileName;  
9  
10- public RealImage(String fileName) {  
11     this.fileName = fileName;  
12     loadFromServer();  
13 }  
14  
15- private void loadFromServer() {  
16     System.out.println("Loading image from remote server: " + fileName);  
17 }  
18  
19- @Override  
20- public void display() {  
21     System.out.println("Displaying image: " + fileName);  
22 }
```

The output window shows the following execution results:

```
Image object created.  
  
First Display:  
Loading image from remote server: nature.jpg  
Displaying image: nature.jpg  
  
Second Display:  
Displaying image: nature.jpg  
  
=== Code Execution Successful ===
```

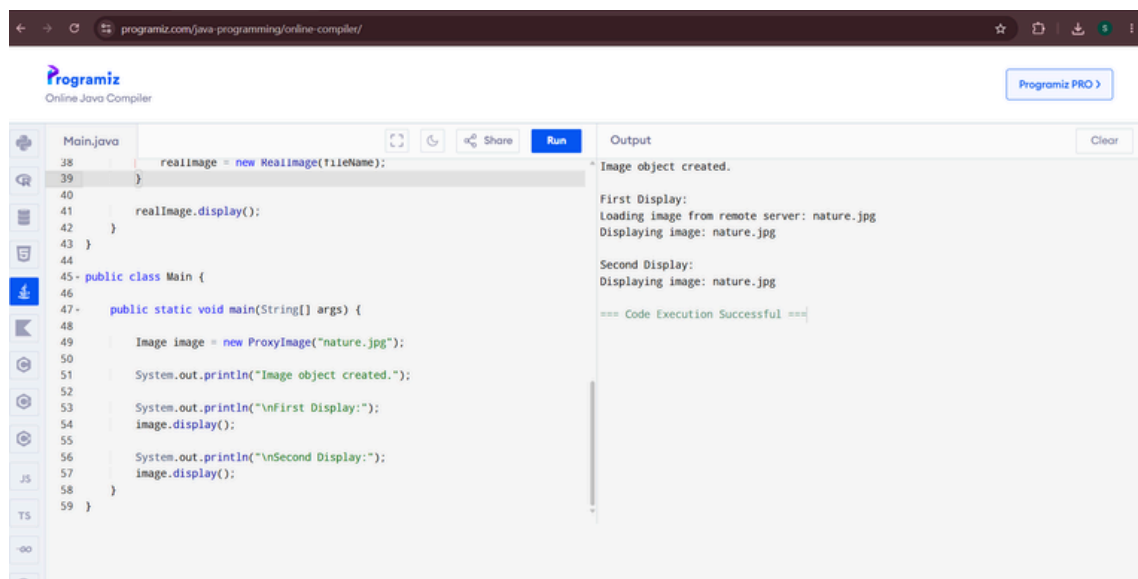


The screenshot shows the Programiz Online Java Compiler interface. The code editor displays the following Java code:

```
24  
25- class ProxyImage implements Image {  
26  
27     private RealImage realImage;  
28     private String fileName;  
29  
30- public ProxyImage(String fileName) {  
31     this.fileName = fileName;  
32 }  
33  
34- @Override  
35- public void display() {  
36  
37     if (realImage == null) {  
38         realImage = new RealImage(fileName);  
39     }  
40  
41     realImage.display();  
42 }  
43 }  
44
```

The output window shows the following execution results:

```
Image object created.  
  
First Display:  
Loading image from remote server: nature.jpg  
Displaying image: nature.jpg  
  
Second Display:  
Displaying image: nature.jpg  
  
=== Code Execution Successful ===
```



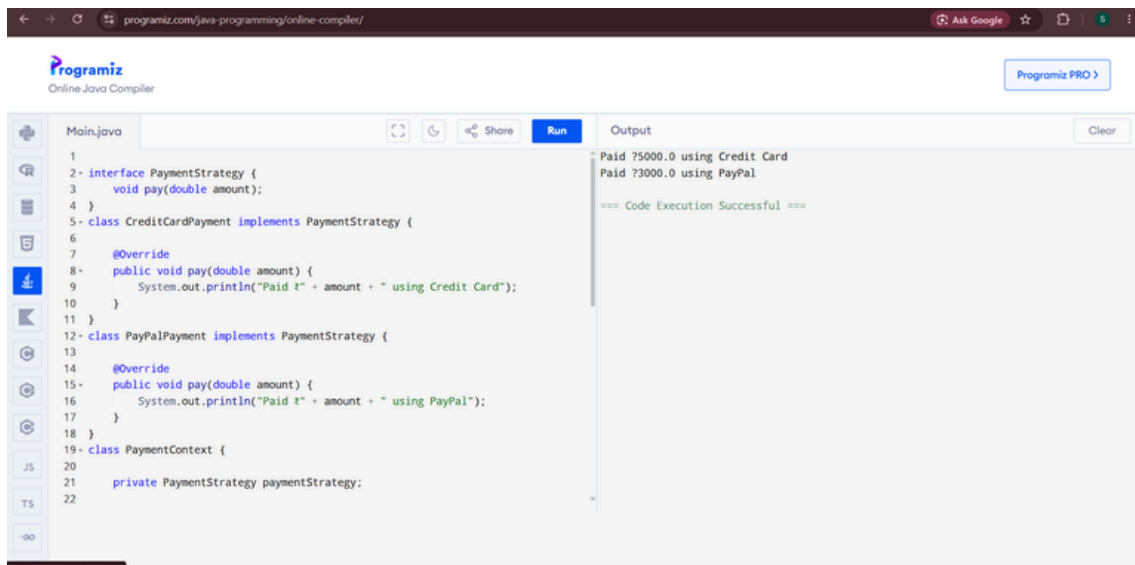
The screenshot shows the Programiz Online Java Compiler interface. The code editor displays the following Java code:

```
38     realImage = new RealImage(fileName);  
39 }  
40  
41     realImage.display();  
42 }  
43 }  
44  
45- public class Main {  
46  
47- public static void main(String[] args) {  
48  
49     Image image = new ProxyImage("nature.jpg");  
50  
51     System.out.println("Image object created.");  
52  
53     System.out.println("\nFirst Display:");  
54     image.display();  
55  
56     System.out.println("\nSecond Display:");  
57     image.display();  
58 }  
59 }
```

The output window shows the following execution results:

```
Image object created.  
  
First Display:  
Loading image from remote server: nature.jpg  
Displaying image: nature.jpg  
  
Second Display:  
Displaying image: nature.jpg  
  
=== Code Execution Successful ===
```

8. Strategy Pattern

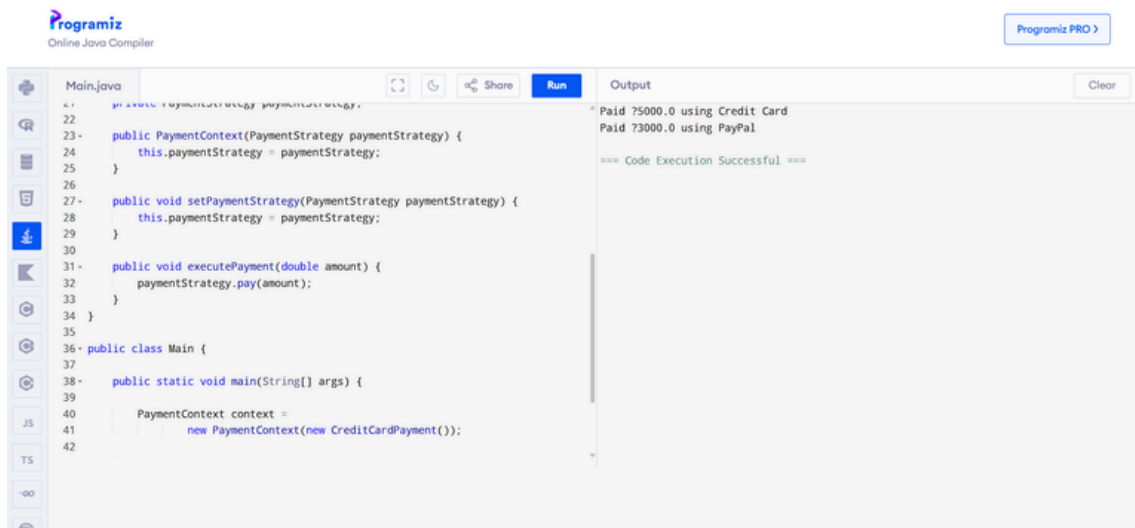


The screenshot shows the Programiz Online Java Compiler interface. The code editor contains the following Java code:

```
1
2- interface PaymentStrategy {
3     void pay(double amount);
4 }
5- class CreditCardPayment implements PaymentStrategy {
6
7     @Override
8     public void pay(double amount) {
9         System.out.println("Paid ₹" + amount + " using Credit Card");
10    }
11 }
12- class PayPalPayment implements PaymentStrategy {
13
14     @Override
15     public void pay(double amount) {
16         System.out.println("Paid ₹" + amount + " using PayPal");
17    }
18 }
19- class PaymentContext {
20
21     private PaymentStrategy paymentStrategy;
22 }
```

The output window shows the results of the code execution:

```
Paid 75000.0 using Credit Card
Paid 73000.0 using PayPal
=== Code Execution Successful ===
```

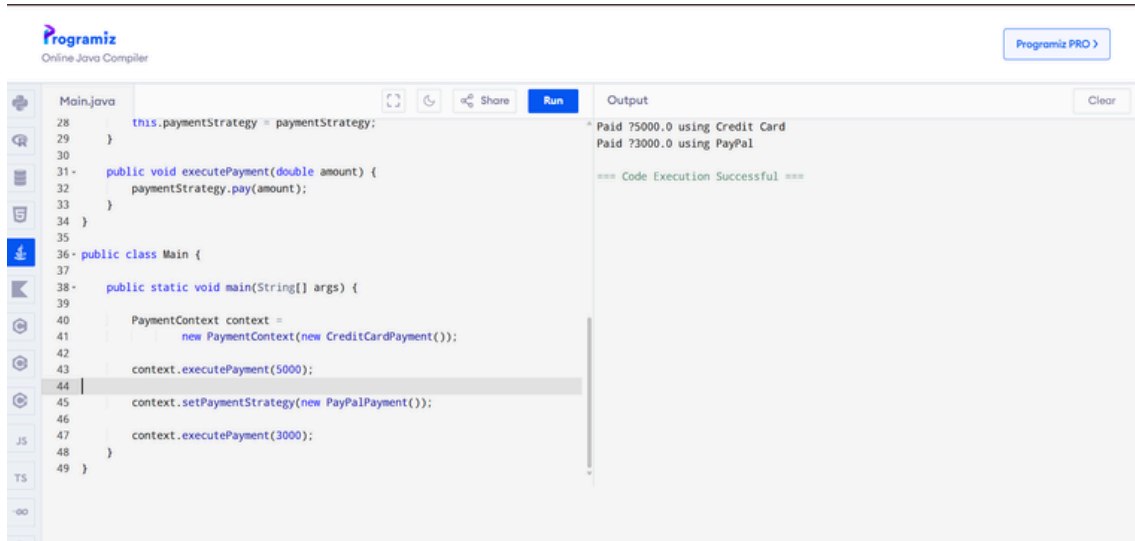


The screenshot shows the Programiz Online Java Compiler interface. The code editor contains the following Java code:

```
22- private PaymentStrategy paymentStrategy;
23- public PaymentContext(PaymentStrategy paymentStrategy) {
24     this.paymentStrategy = paymentStrategy;
25 }
26
27- public void setPaymentStrategy(PaymentStrategy paymentStrategy) {
28     this.paymentStrategy = paymentStrategy;
29 }
30
31- public void executePayment(double amount) {
32     paymentStrategy.pay(amount);
33 }
34 }
35
36- public class Main {
37
38     public static void main(String[] args) {
39
40         PaymentContext context =
41             new PaymentContext(new CreditCardPayment());
42     }
```

The output window shows the results of the code execution:

```
Paid 75000.0 using Credit Card
Paid 73000.0 using PayPal
=== Code Execution Successful ===
```



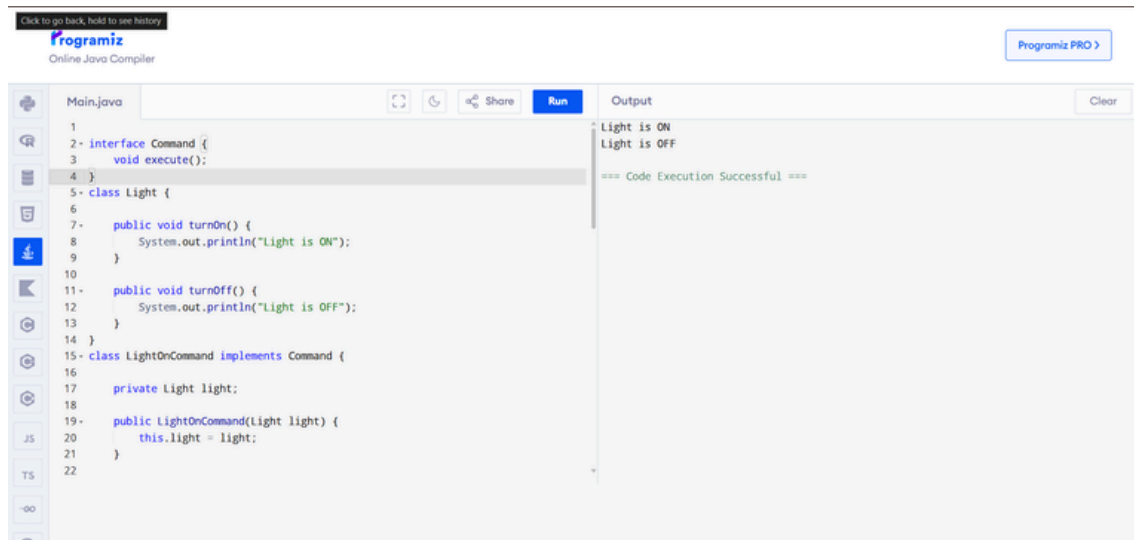
The screenshot shows the Programiz Online Java Compiler interface. The code editor contains the following Java code:

```
28     this.paymentStrategy = paymentStrategy;
29 }
30
31- public void executePayment(double amount) {
32     paymentStrategy.pay(amount);
33 }
34 }
35
36- public class Main {
37
38     public static void main(String[] args) {
39
40         PaymentContext context =
41             new PaymentContext(new CreditCardPayment());
42
43         context.executePayment(5000);
44
45         context.setPaymentStrategy(new PayPalPayment());
46
47         context.executePayment(3000);
48     }
49 }
```

The output window shows the results of the code execution:

```
Paid 75000.0 using Credit Card
Paid 73000.0 using PayPal
=== Code Execution Successful ===
```


9. Command Pattern

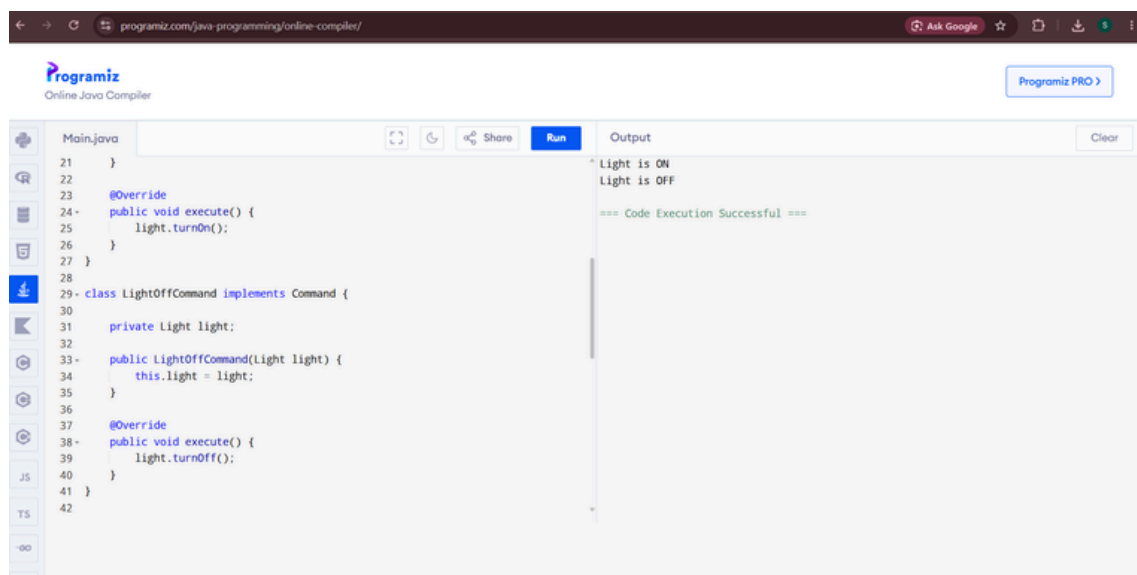


This screenshot shows the first implementation of the Command Pattern in the Programiz Online Java Compiler. The code defines a `Command` interface with an `execute()` method. A `Light` class has `turnOn()` and `turnOff()` methods that print their status. A `LightOnCommand` class implements `Command` and holds a `Light` object. The `Main` class (partially visible) would use these to demonstrate the pattern.

```
1 interface Command {
2     void execute();
3 }
4
5 class Light {
6
7     public void turnOn() {
8         System.out.println("Light is ON");
9     }
10
11    public void turnOff() {
12        System.out.println("Light is OFF");
13    }
14 }
15 class LightOnCommand implements Command {
16     private Light light;
17
18     public LightOnCommand(Light light) {
19         this.light = light;
20     }
21 }
22
```

Output:

```
Light is ON
Light is OFF
=== Code Execution Successful ===
```

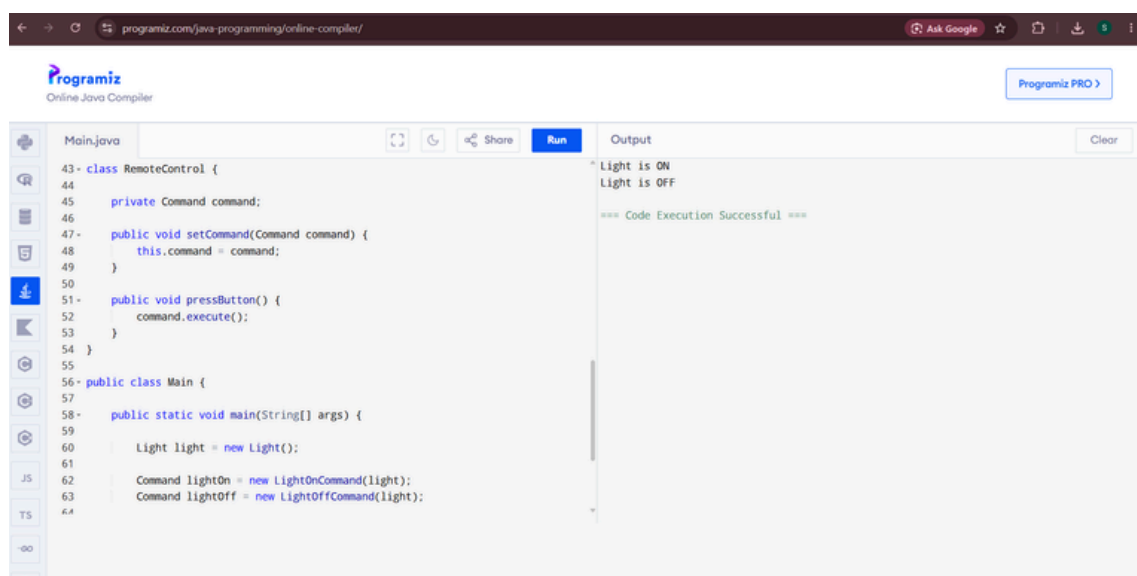


This screenshot shows the second implementation of the Command Pattern. It introduces a `LightOffCommand` class, which also implements the `Command` interface and holds a `Light` object. The `execute()` method in `LightOffCommand` calls `light.turnOff()`. The `Main` class (partially visible) would use these to demonstrate the pattern.

```
21 }
22
23 @Override
24 public void execute() {
25     light.turnOn();
26 }
27 }
28
29 class LightOffCommand implements Command {
30     private Light light;
31
32     public LightOffCommand(Light light) {
33         this.light = light;
34     }
35
36     @Override
37     public void execute() {
38         light.turnOff();
39     }
40 }
41
42
```

Output:

```
Light is ON
Light is OFF
=== Code Execution Successful ===
```



This screenshot shows the third implementation of the Command Pattern. It introduces a `RemoteControl` class that holds a `Command` object and has a `pressButton()` method that calls `command.execute()`. The `Main` class (partially visible) would use these to demonstrate the pattern.

```
43 class RemoteControl {
44     private Command command;
45
46     public void setCommand(Command command) {
47         this.command = command;
48     }
49
50     public void pressButton() {
51         command.execute();
52     }
53 }
54
55 public class Main {
56     public static void main(String[] args) {
57
58         Light light = new Light();
59
60         Command lightOn = new LightOnCommand(light);
61         Command lightOff = new LightOffCommand(light);
62
63     }
64 }
```

Output:

```
Light is ON
Light is OFF
=== Code Execution Successful ===
```

+

🔍

📄

🔒

📌

🏠

🔄

🔄

JS

TS

GO

🔌

Main.java

↺

🔄

🔗

Share

Run

```
52     command.execute();
53 }
54 }
55
56 public class Main {
57
58     public static void main(String[] args) {
59
60         Light light = new Light();
61
62         Command lightOn = new LightOnCommand(light);
63         Command lightOff = new LightOffCommand(light);
64
65         RemoteControl remote = new RemoteControl();
66
67         remote.setCommand(lightOn);
68         remote.pressButton();
69
70         remote.setCommand(lightOff);
71         remote.pressButton();
72     }
73 }
```

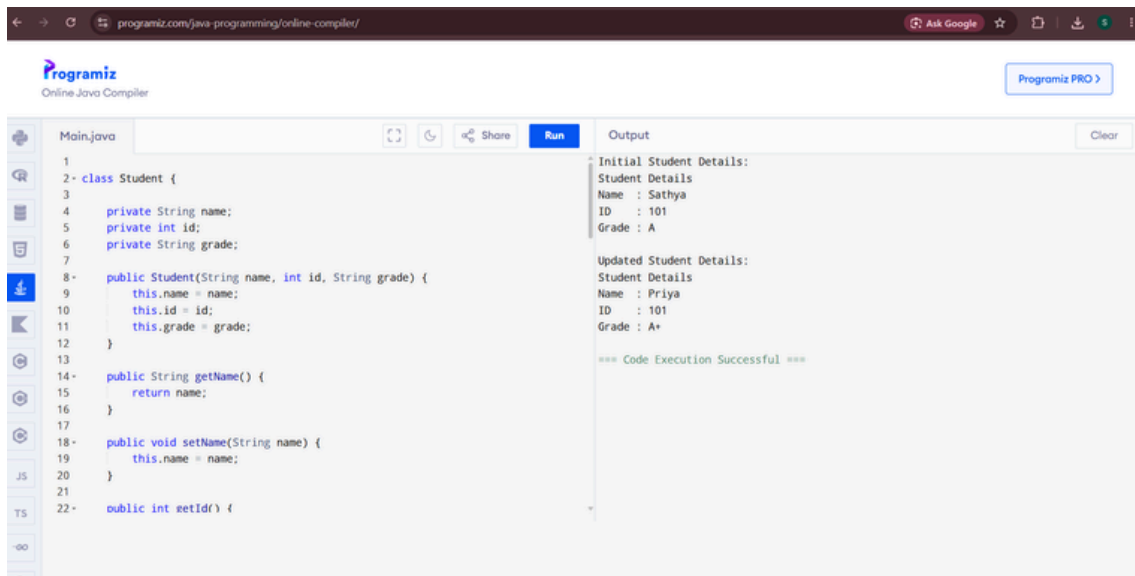
Output

Clear

Light is ON
Light is OFF

=== Code Execution Successful ===

10. MVC Pattern



The screenshot shows the Programiz Online Java Compiler interface. The code editor on the left contains the following Java code for a `Student` class:

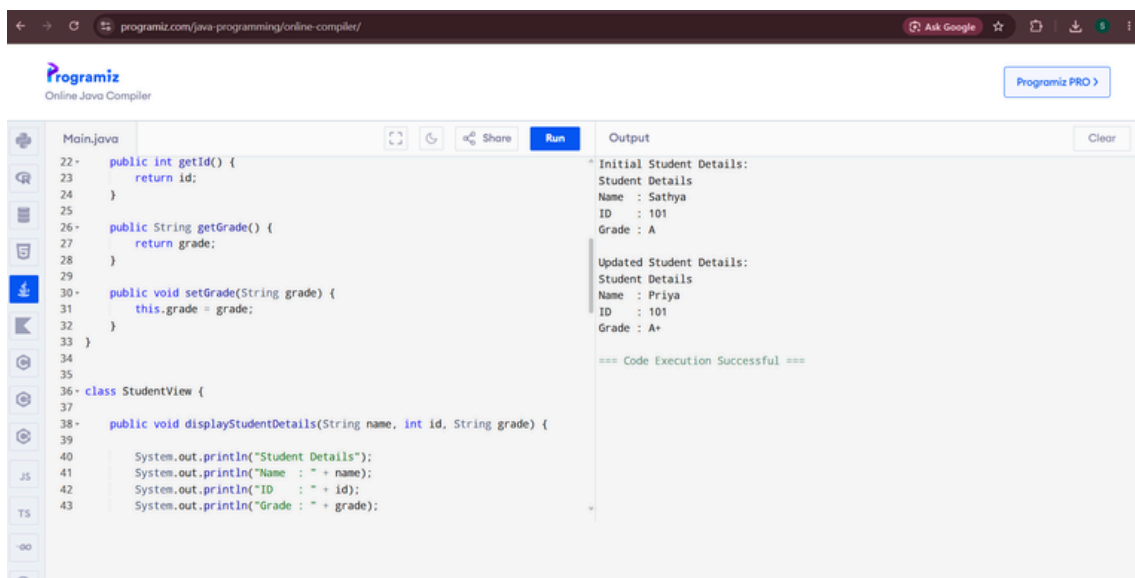
```
1
2- class Student {
3
4     private String name;
5     private int id;
6     private String grade;
7
8- public Student(String name, int id, String grade) {
9     this.name = name;
10    this.id = id;
11    this.grade = grade;
12 }
13
14- public String getName() {
15    return name;
16 }
17
18- public void setName(String name) {
19    this.name = name;
20 }
21
22- public int getId() {
```

The output window on the right displays the results of the code execution:

```
Initial Student Details:
Student Details
Name : Sathya
ID : 101
Grade : A

Updated Student Details:
Student Details
Name : Priya
ID : 101
Grade : A+

=== Code Execution Successful ===
```



The screenshot shows the Programiz Online Java Compiler interface. The code editor on the left contains the following Java code, adding a `StudentView` class:

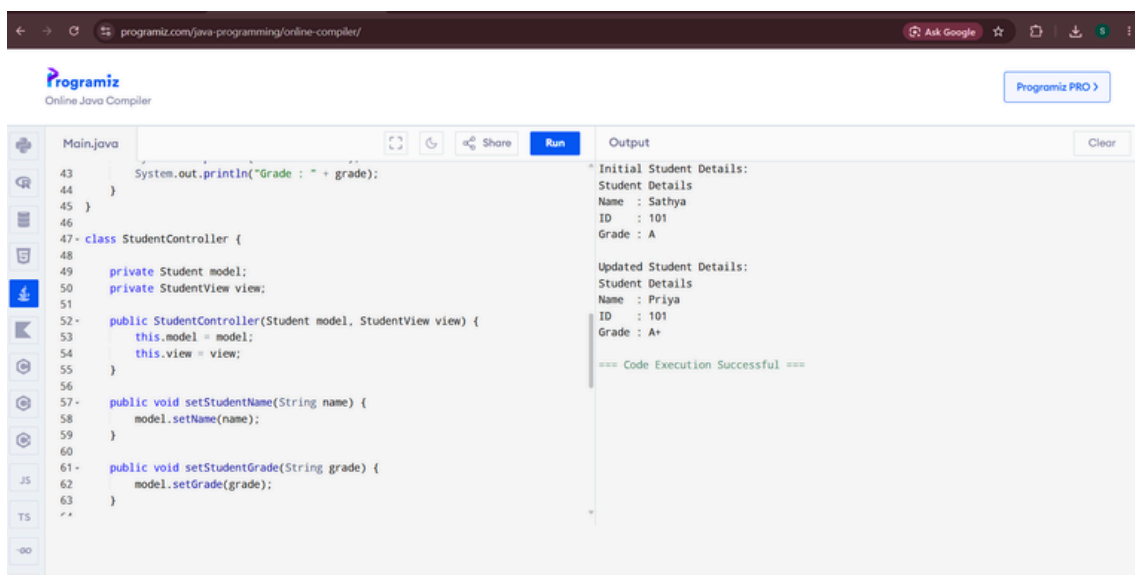
```
22- public int getId() {
23    return id;
24 }
25
26- public String getGrade() {
27    return grade;
28 }
29
30- public void setGrade(String grade) {
31    this.grade = grade;
32 }
33 }
34
35
36- class StudentView {
37
38- public void displayStudentDetails(String name, int id, String grade) {
39
40    System.out.println("Student Details");
41    System.out.println("Name : " + name);
42    System.out.println("ID : " + id);
43    System.out.println("Grade : " + grade);
```

The output window on the right displays the results of the code execution:

```
Initial Student Details:
Student Details
Name : Sathya
ID : 101
Grade : A

Updated Student Details:
Student Details
Name : Priya
ID : 101
Grade : A+

=== Code Execution Successful ===
```



The screenshot shows the Programiz Online Java Compiler interface. The code editor on the left contains the following Java code, adding a `StudentController` class:

```
43    System.out.println("Grade : " + grade);
44 }
45 }
46
47- class StudentController {
48
49    private Student model;
50    private StudentView view;
51
52- public StudentController(Student model, StudentView view) {
53    this.model = model;
54    this.view = view;
55 }
56
57- public void setStudentName(String name) {
58    model.setName(name);
59 }
60
61- public void setStudentGrade(String grade) {
62    model.setGrade(grade);
63 }
64 }
```

The output window on the right displays the results of the code execution:

```
Initial Student Details:
Student Details
Name : Sathya
ID : 101
Grade : A

Updated Student Details:
Student Details
Name : Priya
ID : 101
Grade : A+

=== Code Execution Successful ===
```

The screenshot shows the Visual Studio Code interface during the execution of a Java program. On the left, the Explorer sidebar displays project files: Main.java, JS, TS, and .idea. The Source Control sidebar shows no changes. The Run and Debug sidebar contains icons for running or debugging the application.

The main editor area has two panes:

- Main.java**: Contains the following Java code:

```
64  
65- public void updateView() {  
66     view.displayStudentDetails(  
67         model.getName(),  
68         model.getId(),  
69         model.getGrade()  
70     );  
71 }  
72 }  
73  
74- public class Main {  
75  
76- public static void main(String[] args) {  
77  
78  
79     Student student = new Student("Sathya", 101, "A");  
80  
81  
82     StudentView view = new StudentView();  
83  
84  
85     StudentController controller =
```
- Output**: Displays the output of the program:

```
Initial Student Details:  
Student Details  
Name : Sathya  
ID : 101  
Grade : A  
  
Updated Student Details:  
Student Details  
Name : Priya  
ID : 101  
Grade : A+  
  
*** Code Execution Successful ***
```

Main.java

Run

Clear

```
//
78
79 Student student = new Student("Sathya", 101, "A");
80
81
82 StudentView view = new StudentView();
83
84
85 StudentController controller =
86     new StudentController(student, view);
87
88 System.out.println("Initial Student Details:");
89 controller.updateView();
90
91
92 controller.setStudentName("Priya");
93 controller.setStudentGrade("A+");
94
95 System.out.println("\nUpdated Student Details:");
96 controller.updateView();
97
98 }
```

Output

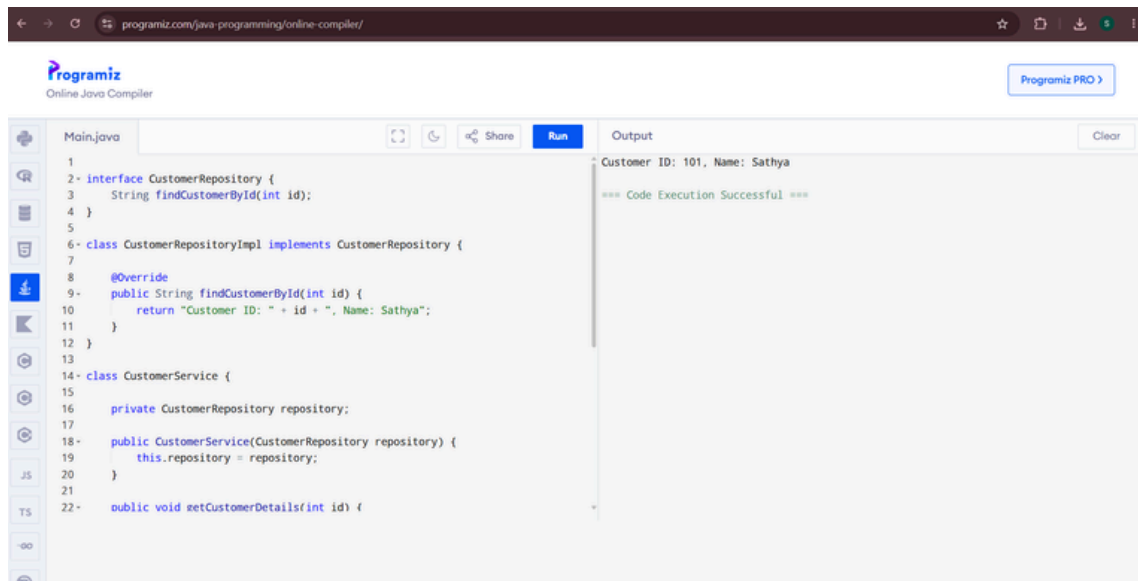
Clear

```
Initial Student Details:
Student Details
Name : Sathya
ID : 101
Grade : A

Updated Student Details:
Student Details
Name : Priya
ID : 101
Grade : A+

=== Code Execution Successful ===
```

11. Dependency Injection

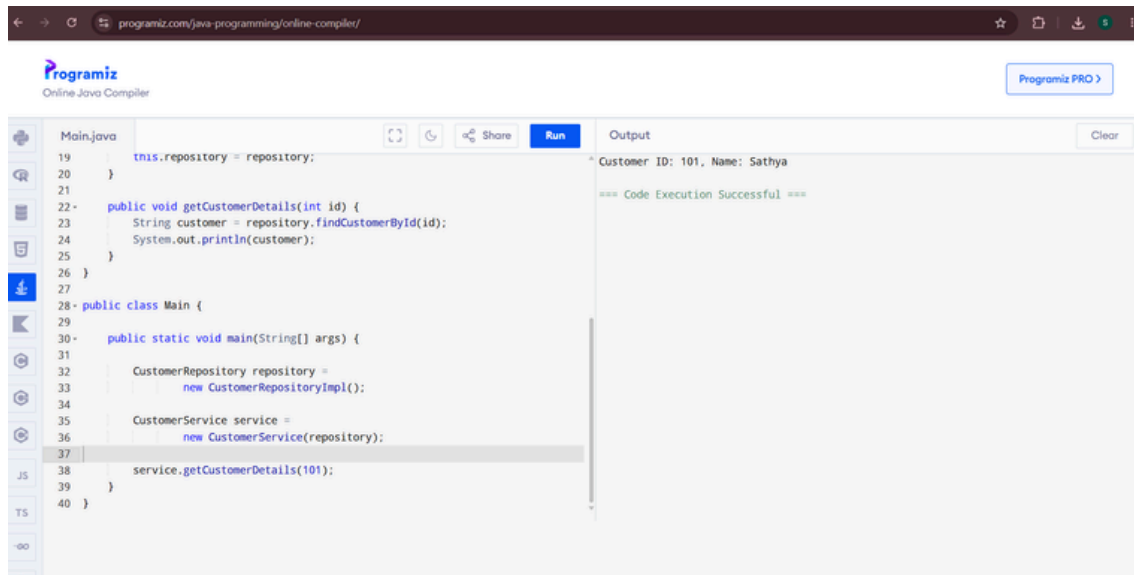


The screenshot shows the Programiz Online Java Compiler interface. The code editor displays the following Java code:

```
1  
2- interface CustomerRepository {  
3   String findCustomerById(int id);  
4 }  
5  
6- class CustomerRepositoryImpl implements CustomerRepository {  
7  
8   @Override  
9- public String findCustomerById(int id) {  
10    return "Customer ID: " + id + ", Name: Sathya";  
11  }  
12 }  
13  
14- class CustomerService {  
15  
16   private CustomerRepository repository;  
17  
18- public CustomerService(CustomerRepository repository) {  
19   this.repository = repository;  
20 }  
21  
22- public void getCustomerDetails(int id) {
```

The output window on the right shows the result of the execution:

```
Customer ID: 101, Name: Sathya  
=== Code Execution Successful ===
```



The screenshot shows the Programiz Online Java Compiler interface with the following Java code:

```
19   this.repository = repository;  
20 }  
21  
22- public void getCustomerDetails(int id) {  
23   String customer = repository.findCustomerById(id);  
24   System.out.println(customer);  
25 }  
26 }  
27  
28- public class Main {  
29  
30   public static void main(String[] args) {  
31  
32     CustomerRepository repository =  
33       new CustomerRepositoryImpl();  
34  
35     CustomerService service =  
36       new CustomerService(repository);  
37  
38     service.getCustomerDetails(101);  
39   }  
40 }
```

The output window on the right shows the result of the execution:

```
Customer ID: 101, Name: Sathya  
=== Code Execution Successful ===
```